Vulnerability Assessment Report

AuSecurity Security Assessment 2026-10-06

AuSecurity
63 Paddington St
Paddington NSW 2021

Flinders University
Sturt Road, Bedford Park SA 5042

October 6, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 3

1 Document Control 4

1.1 Team 4

1.2 List of Changes 4

2 Executive Summary 5

2.1 Overview 5

2.2 Identified Vulnerabilities 5

3 Methodology 7

3.1 Objective 7

3.2 Scope 7

3.3 User Accounts and Permissions 7

4 Findings 8

C1: Exposed osTicket Installer Allows Remote Takeover 8

C2: Unauthenticated Admin Dashboard with Client PII Exposure 9

H1: DOM-Based Cross-Site Scripting via 'name' Parameter 10

M1: Open Redirect on Login 'redirect' Parameter 11

M2: Self-Signed TLS Certificate on Primary Website 12

L1: Directory Listing Enabled on WordPress Uploads 13

L2: Missing Security Headers 14

L3: WordPress Username Enumeration 15

L4: WordPress XML-RPC Enabled with Dangerous Methods 16

I1: Server Software Version Disclosure 17

I2: WordPress Version Disclosure 18

5 Disclaimer 19

A Appendix 20

A.1 Static Appendix Section 20

List of Figures

Figure 1 - Distribution of identified vulnerabilities 6

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date

2 Executive Summary

2.1 Overview

Executive Summary

AuSecurity engaged ClawSec to carry out an authorized, full-scope security assessment of its public-facing infrastructure. Testing covered four in-scope assets - the main marketing website, the WordPress blog, and the osTicket helpdesk - hosted across three OVH Singapore servers, over a single day on 6 October 2026.

The assessment identified **11 findings**: **2 critical, 1 high, 2 medium, 4 low, and 2 informational**.

Two critical issues require immediate attention:

1. **Client data is publicly exposed.** The administrative dashboard can be opened by anyone on the internet without logging in. It contains third-party client records (company names, contact names, and business email addresses) as well as internal business metrics. The only protection is a script that runs in the visitor's browser *after* the sensitive data has already been delivered, so it can be bypassed with trivial effort.
2. **The helpdesk can be taken over before it goes live.** The osTicket helpdesk has not yet been installed, and its setup wizard is publicly reachable. Anyone who reaches it first can complete the installation, register themselves as the administrator, and gain full control of the system.

Additional findings include a cross-site scripting vulnerability on the consultation confirmation page, an open redirect on the login page, a self-signed TLS certificate on the main website, missing security headers, and several information-disclosure issues on the WordPress blog.

The highest-priority actions are to (1) lock down the admin dashboard behind server-side authentication and remove client data from the static pages, and (2) remove or complete the osTicket installer immediately. Full technical detail and remediation guidance for every finding is provided in the body of this report.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Exposed osTicket Installer Allows Remote Takeover	8
C2	9.3	Unauthenticated Admin Dashboard with Client PII Exposure	9
H1	8.1	DOM-Based Cross-Site Scripting via 'name' Parameter	10
M1	6.1	Open Redirect on Login 'redirect' Parameter	11
M2	4.8	Self-Signed TLS Certificate on Primary Website	12
L1	3.7	Directory Listing Enabled on WordPress Uploads	13
L2	3.7	Missing Security Headers	14
L3	3.7	WordPress Username Enumeration	15

#	CVSS	Description	Page
L4	3.7	WordPress XML-RPC Enabled with Dangerous Methods	16
I1	0.0	Server Software Version Disclosure	17
I2	0.0	WordPress Version Disclosure	18

Vulnerability Overview

In the course of this penetration test **2 Critical**, **1 High**, **2 Medium**, **4 Low** and **2 Info** vulnerabilities were identified:

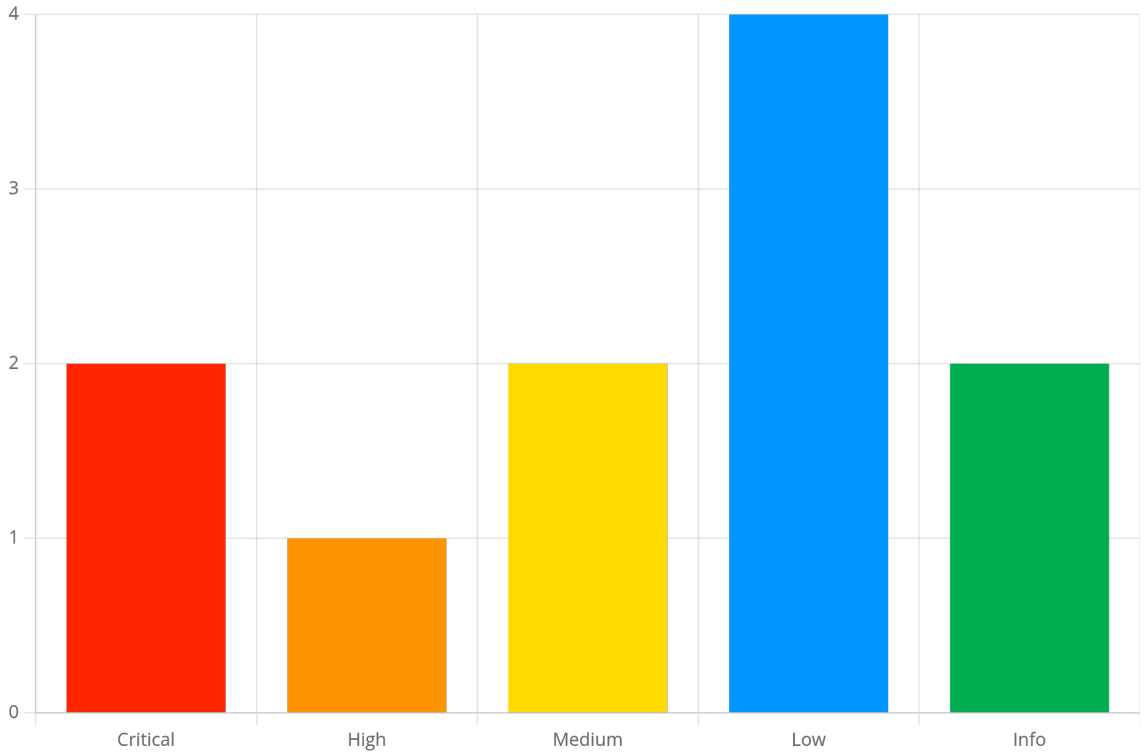


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

Scope

This engagement was a full-scope, black-box security assessment of AuSecurity's public-facing infrastructure, performed from an external (internet) perspective without any provided credentials.

The following in-scope assets were assessed:

- **https://ausecurity.best/** - 51.79.166.148 (static marketing site, nginx/1.22.1, self-signed certificate)
- **https://www.ausecurity.best/** - 51.79.166.148 (same host as the apex domain)
- **https://blog.ausecurity.best/** - 51.79.167.208 (WordPress 7.1.2 blog, Apache 2.4.68 + nginx)
- **https://ticket.ausecurity.best/** - 51.79.166.209 (osTicket v1.18.4 helpdesk, PHP 8.2.33, installer exposed)

Methodology

Authorized full-scope black-box assessment consisting of reconnaissance (DNS, WHOIS, and certificate-transparency review), port and service enumeration, TLS review, and web application testing (broken access control, SQL-injection probing, cross-site scripting, open redirect, and information disclosure) across all four in-scope assets.

Limitations

WHOIS records are privacy-protected, so ownership was inferred from DNS, certificates, and branding. No credentials were provided, so authenticated testing was not performed. A single credential probe was used only for username enumeration (no brute force, in line with the rules of engagement). The osTicket installer was advanced one step solely to confirm exploitability; no installation or configuration was performed.

3.3 User Accounts and Permissions

No user accounts were provided for this engagement.

4 Findings

C1: Exposed osTicket Installer Allows Remote Takeover	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	https://ticket.ausecurity.best/ (51.79.166.209)
References	https://docs.osticket.com/en/latest/Getting%20Started/Installation.html

Overview

The osTicket helpdesk is not yet installed, and its web installer is publicly reachable. Any unauthenticated user can complete the installation wizard and create the administrator account, taking full control of the helpdesk.

Details

https://ticket.ausecurity.best/ returns a 302 redirect to /setup/install.php. The installer (osTicket v1.18.4) is fully reachable and unauthenticated. Posting s=prereq&submit=Continue advances the wizard to step 2 ("Basic Installation"), which exposes the forms to configure the system, create the admin user, and set the database connection - with no authentication, CSRF token, or existing-configuration guard. An attacker who reaches the installer before the business completes setup can install the application against their own database or a database they control, register themselves as administrator, and take over the helpdesk instance (including any future support tickets containing customer data). Note: the prerequisite step was advanced once purely to confirm exploitability; no installation or configuration was performed.

Recommendation

Complete or remove the installer immediately. osTicket's setup directory (/setup/) must be deleted or made inaccessible after installation, and access should be restricted (IP allowlist or authentication) until the application is provisioned. Ensure the instance is installed against a database controlled by AuSecurity before it is exposed to the internet.

C2: Unauthenticated Admin Dashboard with Client PII Exposure	
Score	9.3 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:N
Target	• https://ausecurity.best/dashboard • https://ausecurity.best/dashboard.html (51.79.166.148)
References	https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control.html

Overview

The administrative dashboard is served to any unauthenticated visitor. The full page - including client records and internal business metrics - is present in the static HTML and can be retrieved without logging in.

Details

Requests to /dashboard and /dashboard.html return HTTP 200 with no authentication challenge, no redirect to login, and no server-side access control. The HTML body hard-codes five third-party client records (company name, contact name, and business email address) and internal metrics (47 total enquiries, 12 active clients, 19 assessments YTD, 5 open proposals, revenue YTD $1.2M) plus an activity timeline. The client-side JavaScript contains an auth guard, but it runs only in the browser after the page (including all sensitive data) has already been delivered. Direct HTTP requests (curl, or a browser with JS disabled) bypass it entirely. This discloses regulated personal information (PII) of AuSecurity's clients to anyone on the internet and demonstrates broken access control on the administrative interface.

Recommendation

Enforce authentication and authorization on the server side for all admin content. Do not embed sensitive data in static HTML. Move dashboard data behind an authenticated API (e.g., /api/dashboard) that requires a valid session, and return 401/403 for unauthenticated requests. Remove the client-side-only guard or keep it only as a UX convenience, never as the sole control.

H1: DOM-Based Cross-Site Scripting via 'name' Parameter	
Score	8.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N
Target	https://ausecurity.best/consultation-success.html (51.79.166.148)
References	https://owasp.org/www-community/attacks/DOM_Based_XSS

Overview

The consultation success page reads the 'name' query parameter and injects it into the page using innerHTML without any sanitization, allowing DOM-based cross-site scripting.

Details

The inline script on /consultation-success.html parses window.location.search and executes document.getElementById('user-name').innerHTML = userName using the raw 'name' parameter with no encoding or sanitization. A crafted URL such as /consultation-success.html?name= will execute attacker-controlled JavaScript in the victim's browser in the origin of ausecurity.best. The contact form's /api/contact response also reflects the submitted full_name into the redirect_url that navigates to this page, creating a practical delivery path. Because the login session cookie is scoped to this same origin, exploitation can lead to session hijacking of authenticated admin users.

Recommendation

Use textContent instead of innerHTML for the 'name' value (as is already done for 'company'), or HTML-encode the value before insertion. Validate/sanitize all values read from URL query parameters before they reach DOM sinks.

M1: Open Redirect on Login 'redirect' Parameter	
Score	6.1 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Target	https://ausecurity.best/login (51.79.166.148)
References	https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html

Overview

The login page uses the 'redirect' query parameter as a redirect target after authentication without performing the intended same-origin validation, enabling an open redirect.

Details

In assets/js/main.js, after a successful login the code reads the 'redirect' parameter and assigns it directly to window.location.href. The source contains a comment claiming "only allow same-origin redirects", but no such check exists in the code (no origin comparison, allowlist, or URL parsing). An attacker can craft /login?redirect=https://evil.example.com and, after the victim authenticates, the victim is redirected to the attacker-controlled site. This is commonly abused for credential phishing. The same unsafe pattern also exists in the already- authenticated branch of the login page.

Recommendation

Implement actual same-origin validation (e.g., require the redirect target to start with '/' or match the site origin) before navigating. Prefer a fixed allowlist of permitted post-login destinations.

M2: Self-Signed TLS Certificate on Primary Website	
Score	4.8 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N
Target	https://ausecurity.best/ (51.79.166.148)
References	https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html

Overview

The main website presents a self-signed TLS certificate, causing browser trust warnings and providing no third-party identity assurance.

Details

The certificate served on ausecurity.best:443 has issuer CN=ausecurity.best and subject CN=ausecurity.best (self-signed), valid 2026-09-14 to 2027-09-14. It is not chained to a public CA, so visitors receive security warnings and traffic authentication relies on trust in the self-signed key rather than a trusted CA. The subdomains blog.ausecurity.best and ticket.ausecurity.best use valid Let's Encrypt certificates, so only the apex site is affected. Notably, the web-application still functions over HTTPS and, combined with the missing HSTS header, this weakens transport-security guarantees for the primary domain.

Recommendation

Replace the self-signed certificate with a valid certificate from a public CA (e.g., Let's Encrypt), consistent with the subdomains. Automate renewal and deployment.

L1: Directory Listing Enabled on WordPress Uploads	
Score	3.7 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/wp-content/uploads/ (51.79.167.208)
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/03-Review_Webserver_Metafiles_for_Information_Leakage

Overview

Apache directory indexing is enabled on the WordPress uploads directory, exposing the structure of uploaded content.

Details

GET /wp-content/uploads/ returns an Apache autoindex page ("Index of /wp-content/uploads") listing the 2026/ subdirectory, and /wp-content/uploads/2026/ lists the 09/ and 10/ month directories. This exposes the existence and layout of uploaded files, which can aid further reconnaissance or reveal files not intended to be discoverable.

Recommendation

Disable directory listing (Options -Indexes) for the WordPress directories, and consider placing an empty index file or blocking direct access to wp-content listing.

L2: Missing Security Headers	
Score	3.7 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://ausecurity.best/ • https://blog.ausecurity.best/
References	https://owasp.org/www-project-secure-headers/

Overview

The main website and the WordPress blog omit standard HTTP security headers, reducing the sites' protection against common browser-side attacks.

Details

Responses from https://ausecurity.best/ do not include Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, or Permissions-Policy. The blog (blog.ausecurity.best) likewise omits these headers on its main responses (its wp-login.php does set X-Frame-Options and a CSP frame-ancestors directive). The absence of HSTS is particularly notable on the main site, which is also served with a self-signed certificate. Missing X-Frame-Options/CSP frame-ancestors enables clickjacking on the main marketing pages.

Recommendation

Configure the web server to emit a consistent set of security headers on all responses: Strict-Transport-Security, X-Content-Type-Options: nosniff, X-Frame-Options: DENY (or CSP frame-ancestors 'self'), Referrer-Policy, a Content-Security-Policy, and Permissions-Policy.

L3: WordPress Username Enumeration	
Score	3.7 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/ (51.79.167.208)
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/03-Identity_Management_Testing/04-Testing_for_Account_Enumeration_and_Guessable_User_Account

Overview

The WordPress blog discloses that the username 'admin' exists, both via the REST API and via a distinguishable login error message.

Details

The REST endpoint /wp-json/wp/v2/users is publicly accessible and returns [{"id":1,"name":"admin","slug":"admin",...}], exposing the administrator username. Additionally, submitting log=admin with a wrong password to wp-login.php returns "The password you entered for the username admin is incorrect.", confirming the account exists (as opposed to an "invalid username" message). This assists targeted attacks against the admin account.

Recommendation

Restrict the WordPress REST users endpoint (disable user enumeration) and use generic login error messages ("Invalid username or password") rather than differentiating between invalid username and incorrect password.

L4: WordPress XML-RPC Enabled with Dangerous Methods	
Score	3.7 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/xmlrpc.php (51.79.167.208)
References	https://blog.sucuri.net/2015/10/brute-force-amplification-attacks-against-wordpress-xmlrpc.html

Overview

WordPress XML-RPC is enabled and exposes 78 methods including pingback.ping and system.multicall, which can be abused for amplification and SSRF.

Details

POSTing system.listMethods to /xmlrpc.php returns 200 with a full method listing, including pingback.ping (SSRF / DDoS reflection vector) and system.multicall (brute-force amplification). The pingback functionality allows an attacker to make the blog issue requests to arbitrary hosts, which can be abused for internal network probing or distributed reflection.

Recommendation

Disable XML-RPC if not required (e.g., block /xmlrpc.php at the server level or use a plugin to disable it). If it must remain, restrict pingback.ping and system.multicall.

I1: Server Software Version Disclosure	
Score	0.0 (Info)
Vector string	N/A
Target	• 51.79.166.148 • 51.79.167.208 • 51.79.166.209
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/03-Review_Webserver_Metafiles_for_Information_Leakage

Overview

Web servers disclose their exact software versions, aiding attackers in targeting known-vulnerable versions.

Details

The Server header and service banners disclose exact versions: nginx/1.22.1 (ausecurity.best 80/443, ticket.ausecurity.best), nginx/1.25.5 (ausecurity.best :8080), Apache/2.4.68 (blog.ausecurity.best), PHP/8.2.33 (ticket.ausecurity.best via X-Powered-By), and WordPress 7.1.2 (blog generator meta). OpenSSH 9.2p1 is also exposed on port 22 across hosts.

Recommendation

Set 'server_tokens off;' in nginx, 'ServerTokens Prod' / 'ServerSignature Off' in Apache, and remove the X-Powered-By header (expose_php = Off). Remove or suppress the WordPress generator meta tag.

I2: WordPress Version Disclosure	
Score	0.0 (Info)
Vector string	N/A
Target	https://blog.ausecurity.best/ (51.79.167.208)
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/03-Review_Webserver_Metafiles_for_Information_Leakage

Overview

The blog's HTML discloses the exact WordPress core version via the generator meta tag.

Details

The blog homepage includes and the wp-emoji release script references ver=7.1.2, disclosing the exact core version. Combined with the readme.html and license.txt files being publicly accessible, this simplifies identification of applicable vulnerabilities.

Recommendation

Remove or suppress the WordPress generator meta tag and consider restricting access to readme.html and license.txt.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section